LAB 7 REPORT: PROPERTY-BASED FUZZ TESTING

Course: Software Engineering Lab | Student SRN: PES1UG24CS697 | Topic: Fuzz Testing with Hypothesis

---

1. Learning Outcomes & Objectives

The primary goal of this lab is to use dynamic property-based fuzz testing to uncover hidden edge-case vulnerabilities, uncaught exceptions, and input validation bugs in a target application, and subsequently apply defensive programming practices to harden the implementation. Key objectives include:

- Understanding the purpose, theoretical foundation, and methodology of fuzz testing compared to manual testing.
- Applying property-based fuzz testing in Python using the Hypothesis framework.
- Detecting, reproducing, and analyzing runtime bugs caused by edge-case, boundary, and malformed inputs.
- Refactoring and improving code robustness through defensive input validation and exception prevention.

2. Background: Fuzz Testing & Hypothesis Framework

What is Fuzz Testing? Fuzz testing (or fuzzing) is a dynamic software testing technique where automated test harnesses supply pseudo-random, malformed, or unexpected data to a program to discover bugs, crashes, unhandled exceptions, or security vulnerabilities.

Why Use Fuzzing? Traditional unit tests validate specific, anticipated behaviors ('happy paths' or expected edge cases). In contrast, fuzzers systematically explore expansive input spaces that developers frequently overlook, including unexpected Unicode characters, empty payloads, null bytes, and boundary numeric combinations.

Hypothesis: Hypothesis is a property-based testing library for Python. Rather than asserting fixed input-output pairs, developers write invariants (properties) that must hold true across all valid domain inputs (e.g., 'the function must never crash regardless of string input'). Hypothesis generates diverse test cases conforming to defined strategies (such as st.text()) and automatically shrinks failing inputs to find the minimal reproducing case.

3. Step 1: Environment Setup & Dependencies

The required testing libraries were installed within the dedicated virtual environment:

```
pip install hypothesis
pip install pytest
```

4. Step 2: Initial Execution of Application Script

The assigned application script (spam_scoring.py) implements three core text processing utilities for an email spam filter: sender_domain(email), first_link(text), and shout_ratio(text). The script was initially executed with standard manual inputs:

```
.venv ❯ python spam_scoring.py
==== Running sender_domain ====
Enter a sender email (e.g. offers@Deals.com): PES1UG24CS697@selab.com
Sender Domain: selab.com

==== Running first_link ====
Enter a message containing a link: hi hello https://google.com
First Link: https://google.com

==== Running shout_ratio ====
Enter a message (e.g. FREE prize NOW): FREEE forever FOR REAL
Uppercase Ratio: 0.631578947368421
```

Figure 1: Initial execution of spam_scoring.py with standard manual inputs.

5. Step 3: Designing Property-Based Fuzz Tests with Hypothesis

To evaluate the resilience of the three spam filter functions against unconstrained input, property-based tests were authored in test_spam_scoring.py using Hypothesis strategies. The invariant tested across all three routines is that none of the functions should crash or raise unhandled exceptions when supplied with arbitrary text inputs:

Fuzz Test Suite Implementation (test_spam_scoring.py):

```
from hypothesis import given, strategies as st
from spam_scoring import first_link, sender_domain, shout_ratio

# Test 1: sender_domain
@given(st.text())
def test_sender_domain_no_crash(email):
    sender_domain(email)

# Test 2: first_link
@given(st.text())
def test_first_link_no_crash(text):
    first_link(text)

# Test 3: shout_ratio
@given(st.text())
def test_shout_ratio_no_crash(text):
    shout_ratio(text)
```

6. Step 4: Executing Fuzz Tests & Detecting Vulnerabilities

The test suite was executed using pytest: python -m pytest test_spam_scoring.py -v. Hypothesis immediately exposed critical unhandled exceptions across all three functions during randomized fuzzing.

Analysis of Failure 1: IndexError in sender_domain

Hypothesis generated an empty string (email = "") as the minimal reproducing failure. In the unhardened implementation, the function performed email.split('@')[1] without checking whether an '@' delimiter was present. When given an empty string or any string without '@', split('@') returned a single-element list, triggering an immediate IndexError: list index out of range.

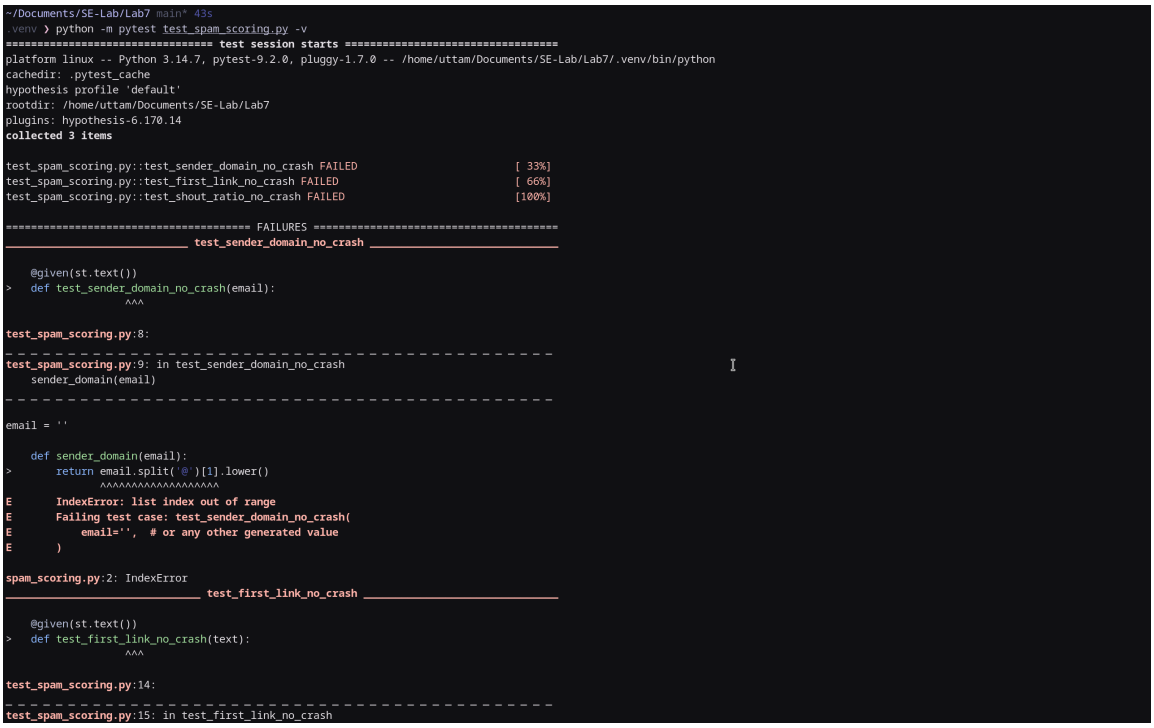


Figure 2: Pytest output showing Hypothesis fuzz test failure (IndexError in sender_domain).

Analysis of Failure 2: ValueError in first_link

For first_link, Hypothesis generated text = "" (as well as arbitrary strings lacking links). The unhardened function relied on text.index('http') to find URLs. In Python, str.index() raises a fatal ValueError: substring not found when the substring is missing, crashing the application for any non-URL message.

Analysis of Failure 3: ZeroDivisionError in shout_ratio

For shout_ratio, Hypothesis discovered text = "" and strings with no alphabetic characters (e.g., whitespace, punctuation, digits). The original code extracted alphabetic letters into a list and returned len(upper) / len(letters). When letters is empty, the denominator is zero, triggering a fatal ZeroDivisionError: division by zero.



Figure 3: Pytest output showing failures in first_link (ValueError) and shout_ratio (ZeroDivisionError).

7. Step 5: Application Hardening & Defensive Programming

To address all vulnerabilities discovered by Hypothesis, spam_scoring.py was refactored using defensive programming practices:

- **sender_domain:** Added input type validation (isinstance(email, str)) and explicitly verified presence of '@'. Replaced naive splitting with safe rsplit('@', 1), and ensured the extracted domain is non-empty and contains a period ('.'), returning an empty string on invalid inputs.
- **first_link:** Added type verification and replaced .index('http') with .find('http'), which returns -1 safely instead of raising an exception. Bounded the URL slice cleanly to the subsequent space or the end of the string.
- **shout_ratio:** Added type verification and an explicit zero-length check if len(letters) == 0: return 0.0 before performing division, completely preventing ZeroDivisionError.



Figure 4: Execution of hardened spam_scoring.py gracefully handling diverse inputs.

8. Step 6: Test Re-Execution on Hardened Application

Following defensive hardening, the Hypothesis fuzz test suite was re-run: python -m pytest test_spam_scoring.py -v. Hypothesis generated hundreds of pseudo-random strings across diverse Unicode ranges, lengths, and special characters. All three test cases passed with zero failures or crashes.



Figure 5: Hypothesis test re-run showing 100% pass rate across all property-based fuzz tests.

9. Hardened Application Code (spam_scoring.py)

```
def sender_domain(email):
    if not isinstance(email, str) or '@' not in email:
        return ''
    parts = email.rsplit('@', 1)
    domain = parts[1].strip().lower()
    if not domain or '.' not in domain:
        return ''
    return domain

def first_link(text):
    if not isinstance(text, str):
        return None
    start = text.find('http')
    if start == -1:
        return None
    end = text.find(' ', start)
    if end == -1:
        end = len(text)
    return text[start:end]

def shout_ratio(text):
    if not isinstance(text, str):
        return 0.0
    letters = [c for c in text if c.isalpha()]
    upper = [c for c in letters if c.isupper()]
    if len(letters) == 0:
        return 0.0
    return len(upper) / len(letters)
```

10. Conclusion & Key Takeaways

- **Automated Boundary Discovery:** Property-based fuzz testing with Hypothesis uncovered edge cases (empty strings, missing patterns, zero division) that standard happy-path manual testing missed.
- **Input Minimization (Shrinking):** Hypothesis's automated shrinking pinpointed the minimal reproducing failure (""), accelerating root-cause analysis.
- **Defensive Robustness:** Validating input types, using non-throwing query methods (like .find()), and guarding mathematical operations ensure resilience in production.